

# Language modeling: from counting words to transformers

---

Summer School on AI for Economics and Finance · Day 3, Lecture 1

Anna Smirnova

Figures: Zhang et al., *Dive into Deep Learning* (d2l.ai, CC BY-SA); Raschka, *Build a Large Language Model (From Scratch)* (Manning, 2024). Used for teaching, with attribution.

# The plan: seven eras, one idea each

---

|   | era         | idea                                          | what it fixed                    |
|---|-------------|-----------------------------------------------|----------------------------------|
| 1 | 1970s–      | text as counts (TF-IDF)                       | text → numbers at all            |
| 2 | 1990s–2000s | statistical language models (n-grams)         | fluency; speech and translation  |
| 3 | 2003–2013   | word vectors (Bengio, word2vec)               | meaning shared across words      |
| 4 | 2014–2016   | recurrent nets, neural translation, attention | order, long context              |
| 5 | 2017        | attention and the transformer                 | context-dependent meaning; scale |
| 6 | 2018–2019   | pretraining; BERT vs GPT                      | one model, many tasks            |
| 7 | now         | what to use for what                          |                                  |

Same question all the way through: **how do we turn a piece of text into something we can compute with, and what did each generation lose?**

# 1. Text as counts

---

The applied pipeline economists already use, and where it stops

# Text as data: the pipeline

---

Gentzkow, Kelly & Taddy (2019, *JEL*) describe what applied work does:

1. **represent** each document as a vector
2. **map** the vector to what you care about (regression, classification, index)
3. **use** the output in the usual econometrics

Examples: FOMC statements → hawkish/dovish; 10-K risk sections → exposure; news → policy-uncertainty index (Baker, Bloom & Davis 2016); earnings calls → tone (Loughran & McDonald 2011).

This morning is about step 1: **where does the vector come from?**

Our recurring example, from a real 10-K:

"Physical climate risks may materially affect our operations and financial results."

# Bag of words and TF-IDF (1972–today)

Count how often each vocabulary word appears in a document; down-weight words that appear everywhere:

$$\text{tf-idf}(w, d) = \underbrace{c(w, d)}_{\text{count in } d} \cdot \underbrace{\log \frac{N}{n_w}}_{\text{rarity across corpus}}$$

A document becomes a sparse vector in  $\mathbb{R}^V$ . Then: cosine similarity, logistic regression, LASSO, topic models (LDA). Still a **strong baseline** for topic classification and retrieval. Transparent: you can read the coefficients.

What it throws away:

- **order.** "risks may not materialize" = "risks may materialize not"
- **meaning.** *climate* and *environmental* are orthogonal axes; nothing learned about one transfers to the other
- **unseen words.** a new ticker or product is a new dimension with no history

Dictionaries (Loughran–McDonald) fix meaning by hand, for one domain, once.

Everything that follows is an attempt to learn the representation instead of counting it.

## 2. Statistical language models

---

1990s–2000s: predict the next word, and a metric to keep score

# A language model is a probability distribution over text

---

The probability of a sentence factorizes exactly, no assumption needed (the chain rule):

$$p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t \mid x_1, \dots, x_{t-1})$$

So a model of text = a model of **one conditional at a time**:

"Physical climate risks may materially  
—"

Each factor is a classification with  $V$  classes: input = the words so far, output = a probability for each possible next word.

Why anyone wanted this in 1990:

- **speech recognition**: "recognize speech" vs "wreck a nice beach" — acoustics can't tell; the LM can
- **machine translation**: of many candidate outputs, which one reads like English?
- **spelling, OCR, typing prediction**

The LM was the *fluency* component of every system for twenty years.

## The head: one read-in, one read-out, for every model today

$$\underbrace{x_{<t}}_{\text{token IDs}} \xrightarrow{E} \text{vectors} \xrightarrow{\text{context map}} h_t \in \mathbb{R}^d \xrightarrow{U} z_t \in \mathbb{R}^V \xrightarrow{\text{softmax}} p_t$$

- **embedding**  $E$  ( $V \times d$ ): ID  $\rightarrow$  row of a table. Read-in.
- **context map**: prefix  $\rightarrow$  one vector  $h_t$ . *The only part that changes today*: a count table, an MLP, a recurrence, a transformer.
- **unembedding**  $U$  ( $d \times V$ ): dot  $h_t$  with every token's vector. Read-out. Often  $U = E^\top$ .
- **softmax**: scores  $\rightarrow$  a distribution (day 1).

One gradient, with  $\ell = -\log p_t[y]$ :

$$\nabla_{z_t} \ell = p_t - e_y$$

predicted minus observed. Read it as chips on a table: every word you bet on that did not come up loses chips in proportion to your bet; the word that came up gains the difference. That correction is passed back, by the chain rule, to everything that produced the bet — down to the vectors of the context words.

Embedding and unembedding are the real inverse pair: vocabulary in, vocabulary out. Every model keeps both ends; they differ in the middle box.



# The n-gram model

An **n-gram** is a run of  $n$  consecutive tokens. On our sentence:

- bigrams ( $n=2$ ): *Physical climate, climate risks, risks may, may materially, ...*
- trigrams ( $n=3$ ): *Physical climate risks, climate risks may, ...*

**The model.** Assume the next word depends only on the last  $n-1$  words. For a bigram model:

$$p(x_1, \dots, x_T) \approx \prod_t p(x_t \mid x_{t-1})$$

The parameters are a **table**: one row per context word, one column per next word, each cell a probability. A sentence's probability is a chain of table lookups:

$$p(\text{climate} \mid \text{Physical}) \cdot p(\text{risks} \mid \text{climate}) \cdot p(\text{may} \mid \text{risks}) \cdots$$

**Estimation.** Maximum likelihood for a multinomial is *count and normalize*:

$$\hat{p}(w_t \mid w_{t-1}) = \frac{c(w_{t-1}, w_t)}{c(w_{t-1})}$$

| $c(\cdot, \cdot)$ | risks | may | change |
|-------------------|-------|-----|--------|
| climate           | 180   | 6   | 40     |
| material          | 27    | 119 | 3      |

$$\hat{p}(\text{risks} \mid \text{climate}) = 180/226 = 0.80.$$

**Three problems, all structural**

1. **window**: nothing beyond  $n-1$  words exists
2. **sparsity**:  $V^2 = 2.5 \times 10^9$  cells, most never seen; each extra  $n$  costs another factor of  $V$
3. **no sharing**: every cell is its own parameter — *climate risks* teaches nothing about *environmental risks*

# The score: cross-entropy and perplexity

For an observed token  $y$  and a predicted distribution  $p$ :  $\ell = -\log p(y)$ .

Average over held-out text = **cross-entropy**;  
perplexity  $= e^{\bar{\ell}}$  = "the number of equally likely words the model was choosing among".

| model                       | perplexity on filings |
|-----------------------------|-----------------------|
| uniform over the vocabulary | ~13,000               |
| bigram                      | ~70                   |
| smoothed trigram            | ~40                   |

Why this loss (day 1): it is the negative log-likelihood, so minimizing it is maximum likelihood; it is a **proper scoring rule** — no gain from misreporting uncertainty; and it has a floor,  $\mathbb{E}[-\log p_{\theta}] = H(p^*) + \text{KL}(p^* \| p_{\theta})$ : training shrinks the KL, the entropy of language stays.

**A next-word probability is a probability over strings.**

Ask "is this firm exposed? yes/no" and read  $p(\text{"yes"}) = 0.6$ : that is how often *text like this* continues with "yes" — not how often such firms are exposed.

Take 100 filings scored 0.6. If 30 are exposed, the text-to-world map is  $0.6 \rightarrow 0.3$ . Estimate that map on labelled data before using the number as a probability. Rankings usually survive; numbers do not. (Lecture 2.)

# What a modern model says about our sentence

GPT-2 (2019), reading one token at a time. Numbers are real.

| context ends with     | actual next | $p(\text{actual})$ | bits | model's top 3                 |
|-----------------------|-------------|--------------------|------|-------------------------------|
| Physical              | climate     | 0.000              | 14.9 | \n, , , .                     |
| ... climate risks     | may         | 0.015              | 6.1  | are, , , .                    |
| ... risks may         | materially  | 0.000              | 12.8 | be, include, also             |
| ... may materially    | affect      | <b>0.216</b>       | 2.2  | <b>affect</b> , increase, and |
| ... affect our        | operations  | 0.011              | 6.5  | business, ability, results    |
| ... financial results | .           | <b>0.603</b>       | 0.7  | ., , , ."                     |

Average 4.2 nats/token → **perplexity 68** on a sentence it has never seen, with a 40-token window instead of 2.

# 3. Distributed representations

---

2003–2013: replace the table with a function, and words become vectors

## Bengio's neural language model (2003): replace the table with a function

---

Give every word a vector  $\mathbf{e}_w \in \mathbb{R}^d$ , a row of a  $V \times d$  table  $E$ . Concatenate the vectors of the last  $n-1$  words, one hidden layer, softmax over  $V$ :

$$p(w_t \mid \text{context}) = \text{softmax}(U \tanh(W[\mathbf{e}_{w_{t-2}}; \mathbf{e}_{w_{t-1}}] + b))$$

- a day-1 MLP with the head from slide 8
- parameters grow with  $d$  and  $V$ , not  $V^n$
- the gradient flows into  $E$ : **the word vectors are learned from prediction**

**What it buys: sharing.** Similar words get similar vectors, so evidence about *climate* moves *environmental*.

Measured on 14M tokens of filings: delete every "unfunded → plans" from training. The count table now says 0. The neural model still ranks *plans* **4th of 20,000** after *unfunded* — because *unfunded* sits next to *underfunded*, *contributory*, *qualified*.

**What it does not buy:** the window is still  $n-1$  words.

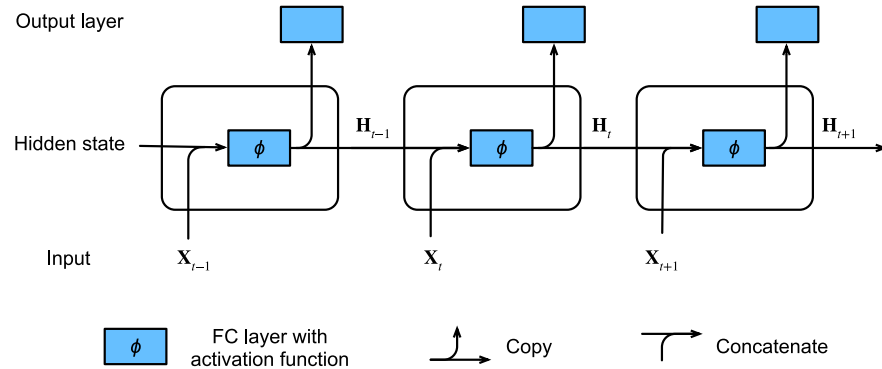
**word2vec** (2013) kept the vectors, dropped the softmax, and ran on billions of words; it factorizes the word–context PMI matrix (Levy & Goldberg 2014). Same information as counting, dense and reusable — the applied workhorse for the next five years.

# 4. Reading in order

---

2014–2016: recurrent networks, neural machine translation, attention

# Recurrent networks: a state that carries the past



Day 1: feed-forward nets are memoryless,  $\hat{y}_t = f(x_t)$ . A **recurrent** net keeps a hidden state and updates it word by word:

$$h_t = \tanh(W_h h_{t-1} + W_x \mathbf{e}_{x_t} + b), \quad p(x_{t+1} | x_{\leq t}) = \text{softmax}(U h_t)$$

- context is, on paper, unbounded:  $h_t$  depends on every earlier word
- one set of weights, applied at every position
- trained with the same cross-entropy at every step

RNN language models (Mikolov 2010) beat smoothed n-grams and became the standard by 2014.

## Why recurrence stalled: the 2017 reason and the 2026 reason

---

**2017: engineering.** Vanishing gradients (day 1): credit from step  $t$  to step  $t - k$  is a product of  $k$  Jacobians and shrinks geometrically — twelve orders of magnitude by lag 50. LSTM gates mostly fix that. But training is **sequential** — step  $t$  needs  $t - 1$  — so an LSTM could not use a GPU the way a transformer can.

That argument is now false: linear RNNs and state-space models (Mamba, RWKV, xLSTM) train in parallel. They still did not win for language.

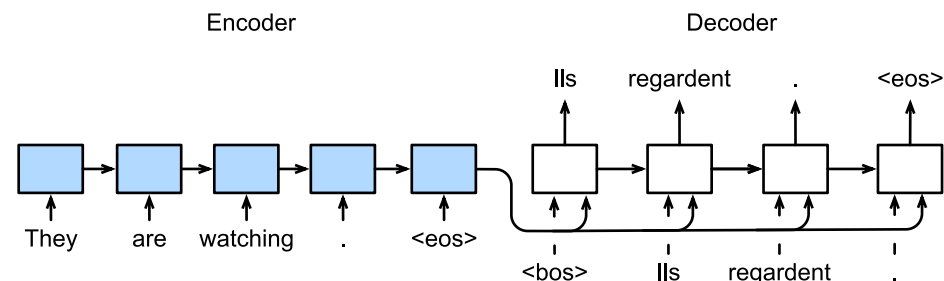
**2026: the state gets muddled.** An RNN has one vector and **every new word writes into it**. Old content is not stored beside new content — it is *blended* with it. After 2,000 updates the specific (a name on line 12) has been averaged into the general. Gates choose what to protect, but at *write* time, before the question exists.

**Attention never blends.** Every word's vector stays put; a query **selects** among them at *read* time, by content. That is the everyday meaning of the word.

**Summarize in advance vs select when needed.** Language needs the second — "it", names, "as defined in Section 3(b)". To match one attention layer on such look-ups, a recurrent state must grow with the text (Jelassi et al.; Arora et al., 2024).



# Translation needs two of them: a reader and a writer (2014)



So far, one RNN predicting the next word of one text. Translation has *two* texts. Sutskever et al. and Cho et al. (2014) used two RNNs:

- an **encoder** reads the source sentence and ends in one vector  $c$  — the reader
- a **decoder** is the language model from the previous slides, started from  $c$ : predict the next target word, append, repeat — the writer

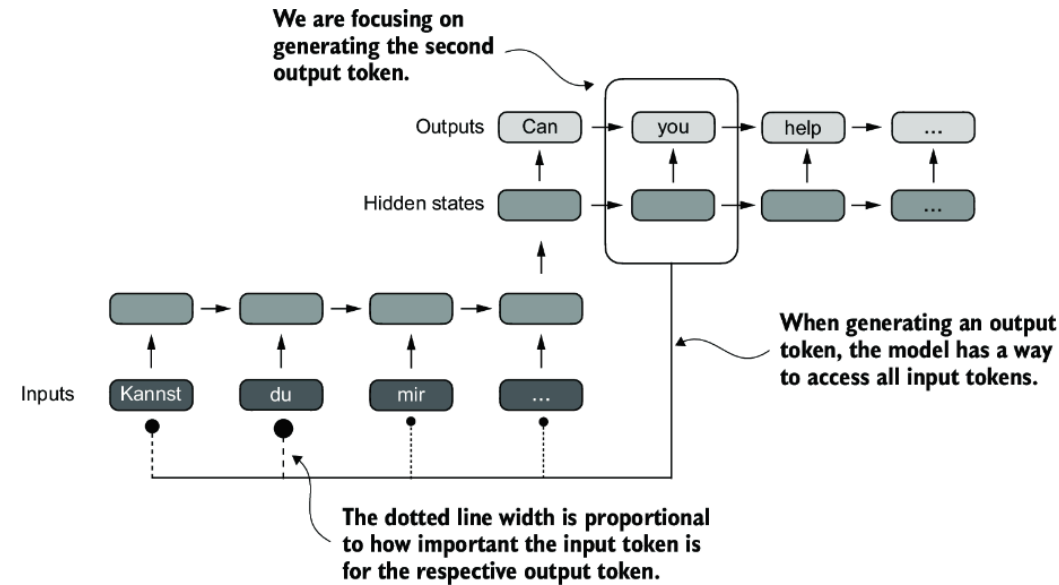
Statistical MT (phrase tables + n-gram LM, 1990–2015) was replaced within two years; Google switched in 2016.

The failure was visible at once: quality **fell with sentence length**. Everything the source says has to pass through one fixed-size vector  $c$  — the muddled state from the previous slide, now with a name.

It was tolerable for a while because source and target line up almost word for word, so a running summary was nearly enough.

Keep the two words: **encoder = reader, decoder = writer**. The transformer, part 5, keeps the names and the roles.

# Attention (2015): look back at everything



Bahdanau, Cho & Bengio: at every output step, compute a weight  $\alpha_{tj}$  for each source position  $j$  from *content* (how well the decoder's state matches the encoder's state there), and feed the decoder a weighted average  $c_t = \sum_j \alpha_{tj} h_j$ .

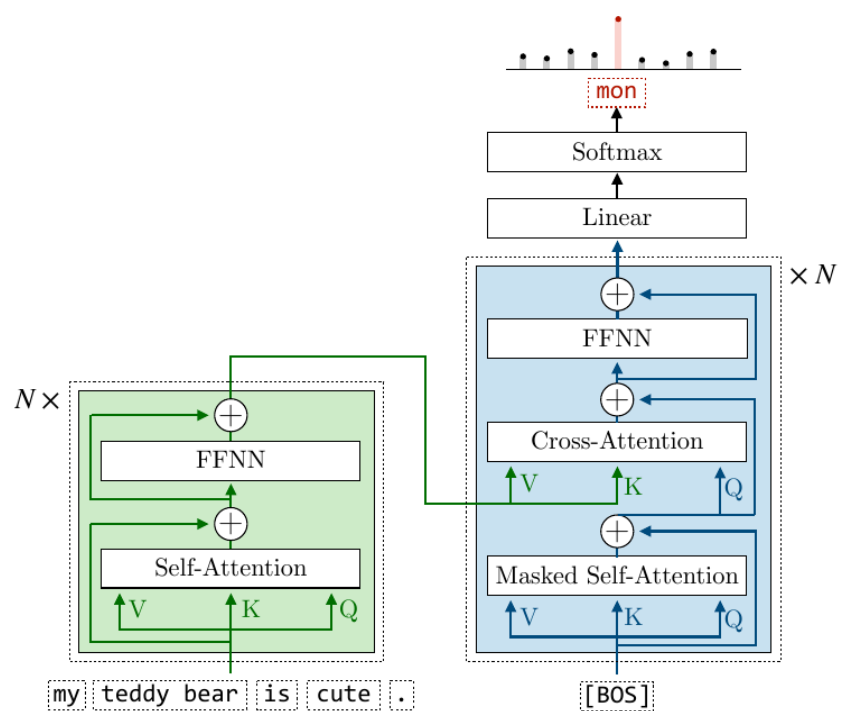
Nothing is discarded in advance; the model **retrieves** what it needs when it needs it. Long-sentence degradation disappeared, and the weights  $\alpha$  turned out to be readable word alignments, learned with no alignment data.

# 5. Attention

---

2017: the one idea to take home today

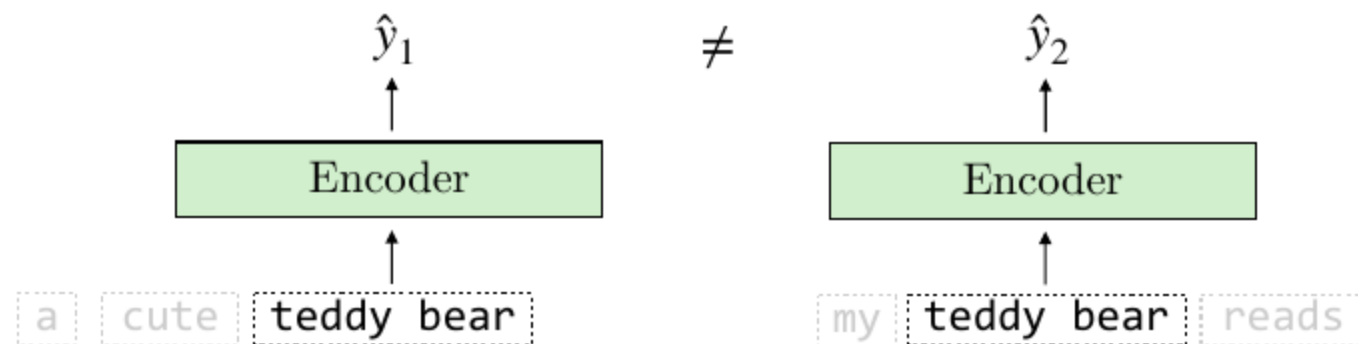
# The whole machine first: the transformer (Vaswani et al. 2017)



A translator. An **encoder** reads the source; a **decoder** writes the target one token at a time; a softmax over the vocabulary sits on top — the same head as every model this morning. Inside each block: a **self-attention** layer and a small feed-forward net, each with its input added back on. Stacked  $N$  times.

The only box you have not seen is *Self-Attention*. The next ten slides open it; then we come back and assemble. GPT keeps the right-hand stack only (minus cross-attention); BERT keeps the left.

# The goal: a word's vector should depend on its neighbours



Everything before this point gives *teddy bear* one vector, wherever it appears.  
word2vec, TF-IDF, a dictionary: one entry per word.

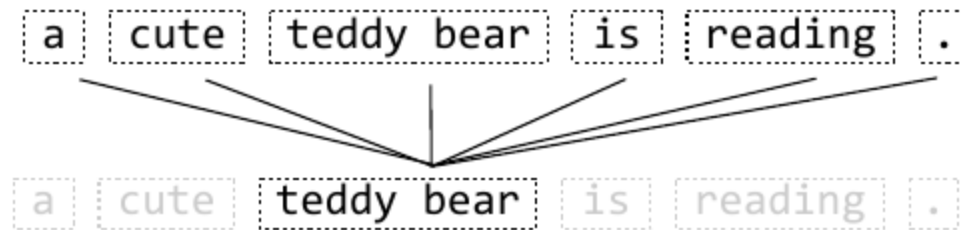
What we want: a vector for *teddy bear* computed **from the sentence it is in**, so "a cute teddy bear" and "my teddy bear reads" give different vectors.

Same for our sentence. *material* in "material weakness" is a legal term; in "raw material" it is a commodity. *risks* after "climate" is one thing; after "credit", another.

The mechanism that does this is **attention**.  
Five slides, one sentence: *a cute teddy bear is reading*.

## Step 0: every word looks at every other word

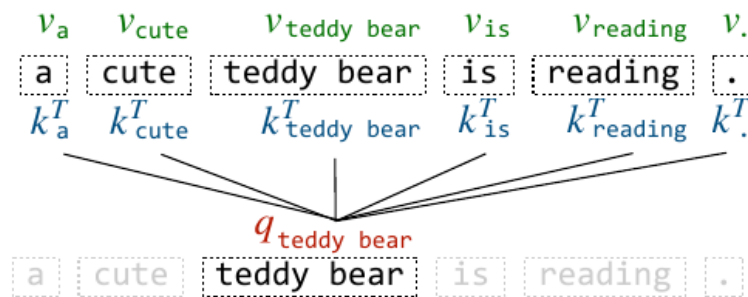
---



To build the new vector for *teddy bear*, let it look at all six tokens in the sentence, including itself, whatever the distance.

The output for *teddy bear* will be a **mix** of the six input vectors. The whole question is: **with what weights?** If the weights were fixed — say, equal — this would be a bag of words again. The weights must depend on the words.

## Step 1: give every word three roles



From each word's vector  $x$ , three vectors, with three **learned** matrices:

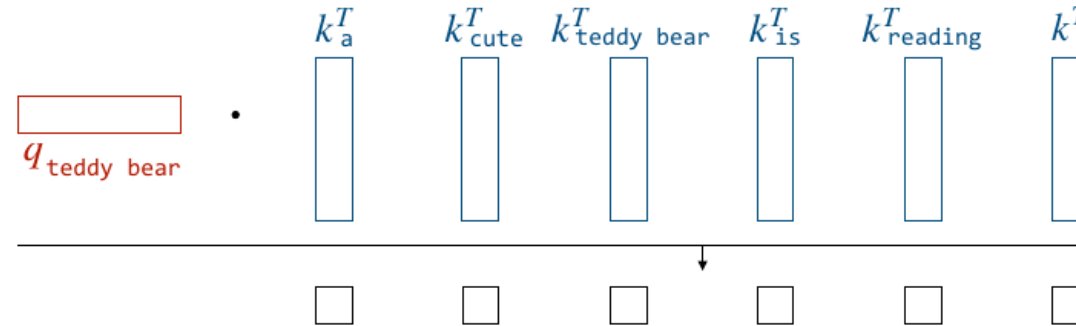
- **query**  $q = W_Q x$  — what this word is *looking for*
- **key**  $k = W_K x$  — what this word *offers* to be matched against
- **value**  $v = W_V x$  — what this word *hands over* if chosen

A library catalogue: search terms = query, index cards = keys, the books = values.

Three views of the **same** word: *teddy bear* has a query (when it looks) and a key and a value (when others look at it).

**Learned, from what, by optimizing what.** Learned:  $W_Q, W_K, W_V$  (plus the embedding table and, later, the MLP). From: raw text, no labels. By optimizing: the next-token cross-entropy of slide 8 — its gradient  $p - e_y$  flows back through softmax, weighted sum and dot products into these matrices. Nothing else on these five slides is learned; the weights  $p_i$  are *computed*.

## Step 2: score every pair, then normalize



Take the query of *teddy bear* and dot it with each word's key: six numbers, "how well does each word match what *teddy bear* is looking for".

Divide by  $\sqrt{d_k}$  (dot products of long vectors are large; this keeps them in a range where the next step behaves), then **softmax** across the six so they are positive and sum to one:

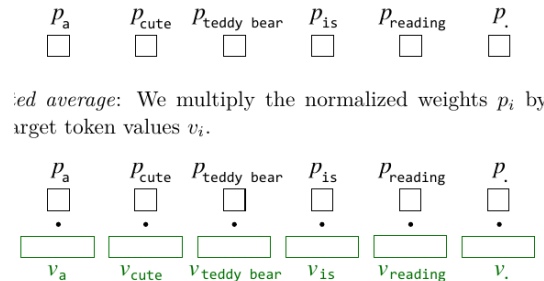
$$p_i = \frac{\exp(q \cdot k_i / \sqrt{d_k})}{\sum_j \exp(q \cdot k_j / \sqrt{d_k})}$$

These six numbers are the **attention weights** for *teddy bear*. They are recomputed for every sentence and every word — they are not parameters.

Softmax is the same operation as the output layer from day 1. Here it is used not to predict a class but to spread a unit of "looking" across the sentence.



## Step 3: take the weighted average of the values



Multiply each word's **value** by its weight and add:

$$z_{\text{teddy bear}} = \sum_i p_i v_i$$

That is the new representation of *teddy bear*, built from the whole sentence, weighted by relevance to it. **That is attention.** Three learned matrices, one dot product, one softmax, one weighted sum.

Another reading: the result **stresses the aspect in which we take the word here**. *teddy bear* has directions for toy, cute, size, owner; "cute" and "reading" pull the *toy-with-a-character* direction forward; "shipping a teddy bear" would pull *object* forward.

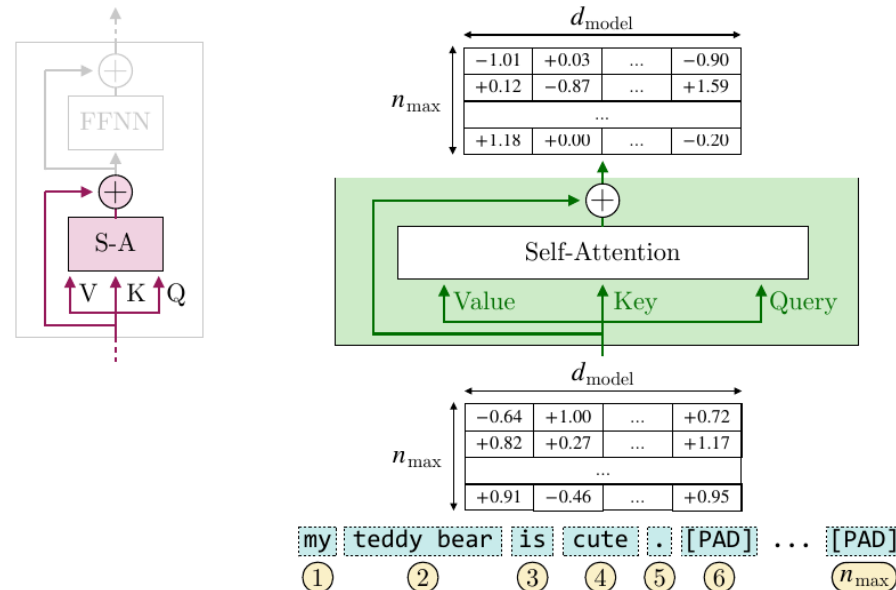
For the econometricians: with kernel  $\kappa(q, k_i) = e^{q \cdot k_i / \sqrt{d_k}}$ , steps 2 + 3 are

$$z = \frac{\sum_i \kappa(q, k_i) v_i}{\sum_i \kappa(q, k_i)}$$

the **Nadaraya–Watson** kernel regression of  $v$  on  $q$ : kernel smoothing in a learned space,  $\sqrt{d_k}$  the bandwidth.

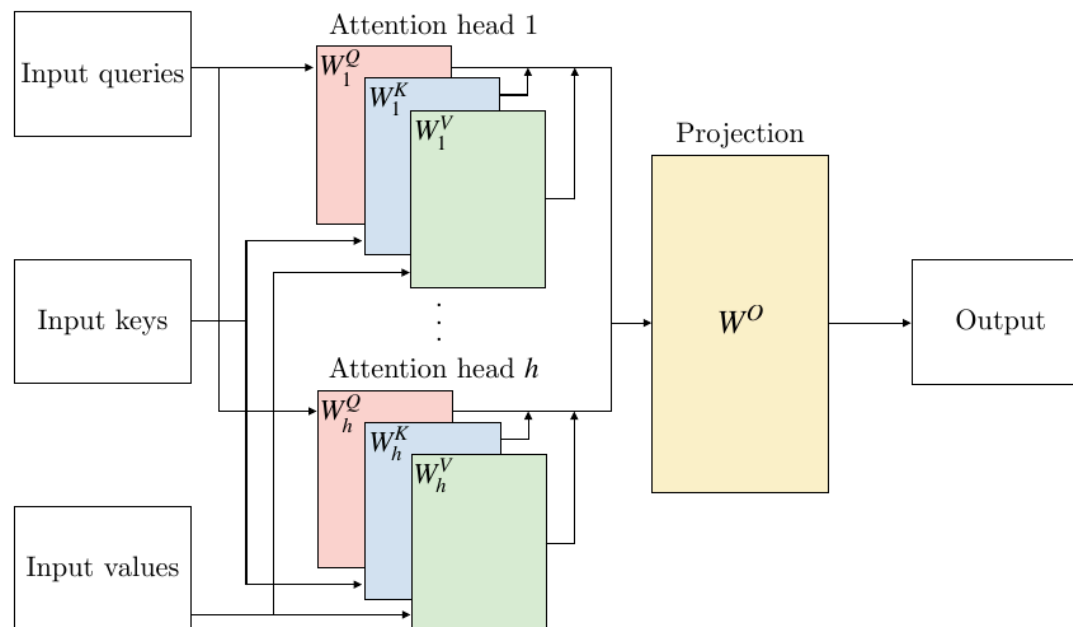
Two checks: all weights  $1/6 \rightarrow$  the sentence average (bag of words). One weight  $= 1 \rightarrow$  a copy of that word. Attention interpolates between averaging and copying, per word, per sentence.

# All words at once: the matrix form, with real numbers



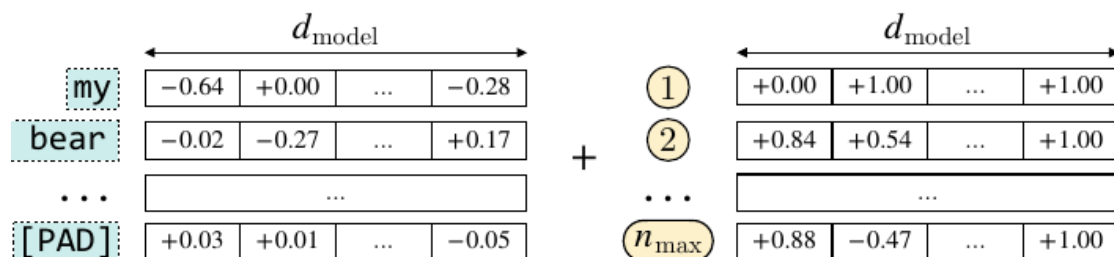
Stack the six queries into  $Q$ , keys into  $K$ , values into  $V$  (each  $n \times d$ ). Every word's output is one row of  $\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k}) V$ .  $QK^\top$  is  $n \times n$  — every word against every word: the  $O(n^2)$  cost, the price of "look at everything".

# Several heads, then a projection



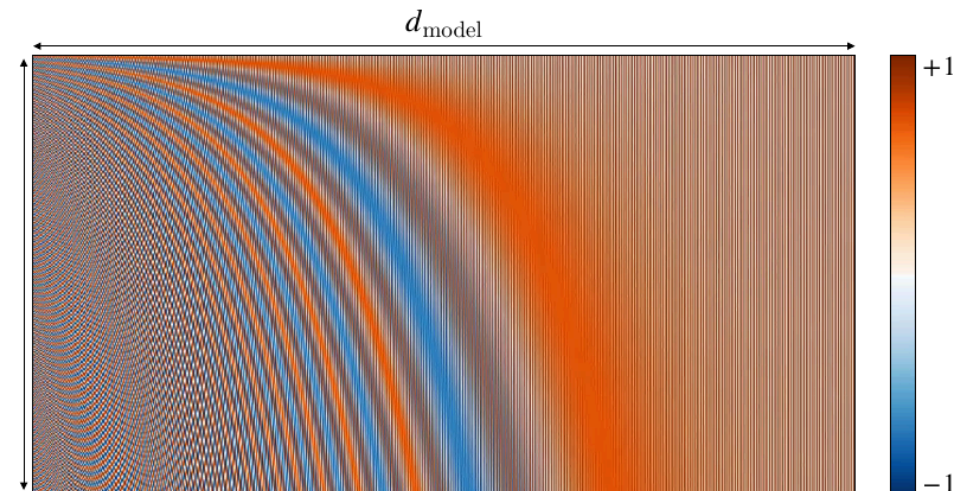
Run  $h$  copies of steps 1–3 in parallel, each with its own  $W_Q$ ,  $W_K$ ,  $W_V$ , concatenate the outputs and project with  $W_O$ . Each head can learn a different notion of relevance: one tracks the subject, another the nearest adjective, another the previous token. GPT-2 small has 12 heads per layer; head "roles" are found after training, not designed.

# What attention forgets: order



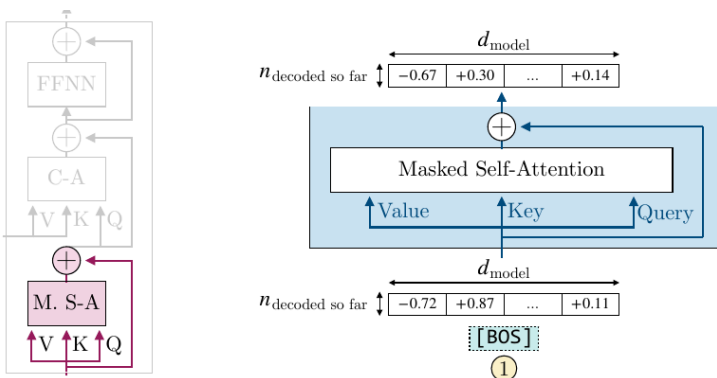
Steps 0–3 never use *where* a word is. Permute the sentence and every output permutes with it: "risks may not materialize" and "risks may materialize not" get the same set of vectors.

Fix: **add a position vector** to each token vector before attention. Learned per position (GPT-2, BERT), or fixed sinusoids —



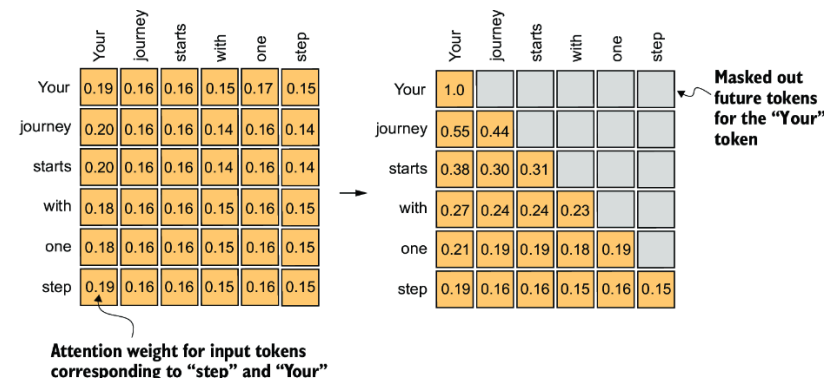
— one row per position, one column per dimension. Nearby positions get similar vectors, so a dot product between two keys carries "how far apart are we". Newer models rotate  $q$  and  $k$  by a position-dependent angle instead (RoPE), with the same effect.

# What attention must not do: peek



For next-word prediction, position  $t$  may use positions  $1..t$  only; otherwise the model sees the answer.

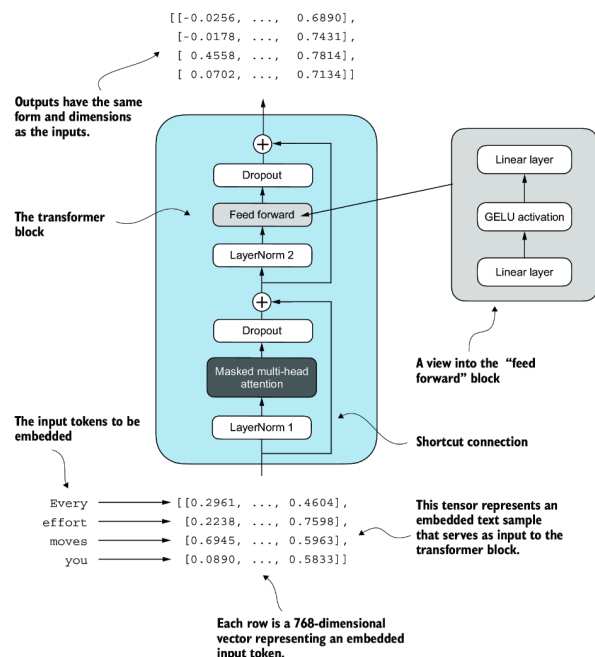
Fix: the **causal mask** — set the upper triangle of  $QK^\top$  to  $-\infty$  before the softmax; those weights become exactly zero. The chain rule from part 2, as a matrix. In finance terms: an as-of date.



**Only because we predict the next word.** To read a whole sentence and label it, leave the mask off — every word sees both sides — and then you must *hide* words to have something to predict. That is BERT, part 6.

Two "masks", one word. The figure's *masked self-attention* = this causal mask, inside attention. BERT's *masked language model* = tokens hidden in the input; its attention is not masked.

# The transformer block: attention, then an MLP, with shortcuts



One block, repeated  $L$  times (12 for GPT-2 small,  $\sim 100$  for the largest models):

$$x \leftarrow x + \text{MultiHeadAttention}(\text{LN}(x))$$

$$x \leftarrow x + \text{MLP}(\text{LN}(x))$$

- attention moves information **between** words
- the MLP (day 1) computes **within** each word, and holds two-thirds of the parameters
- the  $x + \dots$  **shortcut** is why 100 layers train: the gradient has a straight path (the LSTM's carry line, across depth instead of time)

Embeddings + positions at the bottom, a softmax over the vocabulary at the top, and that is GPT.

## Live: watch it run

---

[poloclub.github.io/transformer-explainer](https://poloclub.github.io/transformer-explainer)  
— GPT-2 in the browser.

Type:

Physical climate risks may materially  
affect our operations and

and follow one head through steps 1–3: the Q/K/V projections, the  $n \times n$  score matrix with its empty upper triangle, the softmax, the weighted sum — then the 12 blocks and the final softmax over 50,257 tokens.

Point at:

- which earlier words *affect* attends to
- how the next-token distribution moves when you change the last word
- the temperature slider: ranking fixed, gaps move

3 minutes. Open it in a second tab before the lecture; it downloads a 124M model.

# 6. Pretraining

---

2018–2019: one model trained once, adapted to everything



# The recipe that changed the field

---

Old: collect labels for your task, train a model from scratch on them. Every task starts at zero; text features are hand-made or word2vec.

New (ELMo → GPT → BERT, 2018):

1. **pretrain** a large transformer on the next-word (or masked-word) task over billions of words. No labels needed — the text supplies them.
2. **fine-tune** (or just use) it on your task with a small labeled set.

Why predicting words teaches so much: to do it well you have to learn whatever helps —

| to predict ...                                | you need              |
|-----------------------------------------------|-----------------------|
| "materially <b>affect</b> "                   | collocations, grammar |
| "may not <b>materialize</b> "                 | negation and scope    |
| "based in Cupertino, <b>California</b> "      | facts                 |
| "revenue fell, so margins <b>compressed</b> " | reasoning, in text    |

We have  $10^3$ – $10^5$  labels per task and  $10^{13}$  tokens of text. Spend the compute on the unlimited signal, adapt to the scarce one.

## Encoder or decoder: the same head, two masks, two things to predict

Both are the pipeline from part 2 — embedding  $E \rightarrow$  blocks  $\rightarrow$  unembedding  $U \rightarrow$  softmax — with the same loss and the same gradient  $p - e_y$ . Two switches differ, and they move together:

|                   | GPT (decoder)                                       | BERT (encoder)                                           |
|-------------------|-----------------------------------------------------|----------------------------------------------------------|
| attention mask    | causal ("masked self-attention"): $t$ sees $\leq t$ | none: $t$ sees everything                                |
| what is predicted | the <b>next</b> token, at every position            | the <b>hidden</b> ( [MASK] ) token, at 15 % of positions |
| what it yields    | a text generator                                    | a contextual vector per token                            |

**Why together.** With no mask, "predict the next token" is trivial — position  $t$  can see  $t+1$  — so an encoder must hide its target instead. With a causal mask, hiding is pointless: the future is already invisible.

**Consequences.** GPT gets a gradient at every position, BERT at 15 % ( $7\times$  less supervision per pass). BERT's vectors see both sides: better for *understanding* a fixed text. GPT's last position has seen everything: a *generator*, and a summary at the end.

## What each one was used for

---

### Encoders (BERT family), 2018–

- classification: sentiment, topic, hawkish/dovish
- token labeling: named entities, numbers, dates
- question answering over a passage
- **sentence embeddings**: Sentence-BERT (2019) — the clean 768-dim vector from part 1 is *this*
- domain versions: **FinBERT** (Huang, Wang & Yang 2023, CAR), ClimateBERT

Cheap: fine-tunes on a laptop in minutes; 110M parameters.

### Decoders (GPT family), 2019–

- text generation, summarization, translation
- few-shot: describe the task in the prompt, no fine-tuning (GPT-3, 2020)
- and eventually chat, code, agents — Lecture 2

Expensive; but one model, no task-specific training at all.

For a labeled classification task on short text, a fine-tuned BERT-sized encoder is still competitive and  $\sim 1,000\times$  cheaper than a frontier model. "Use the biggest decoder" is a recommendation, not a theorem.

# Why encoders faded, and where they still live

**Raw BERT is useless for similarity.** Its vectors are trained to predict masked words, not to make similar sentences close. Reimers & Gurevych's motivating number: finding the most similar pair among 10,000 sentences with BERT means 50 million forward passes, **65 hours**. Sentence-BERT fine-tunes BERT in a siamese setup so a single vector per sentence works: **5 seconds**. That fix, not BERT itself, is what runs vector search today.

**Encoders cannot generate.** One decoder serves classification, extraction, summarization and chat; the scale investment went there, and from 2020 the **best "classifier" was often a prompted decoder**

**Where encoders still win (2024–):** ModernBERT revived the family with a modern recipe; for retrieval and labelled classification on short text, a 100–400M encoder is faster, cheaper and often more accurate than a prompted 70B decoder. And the split is blurring: many current embedding models are *decoders* fine-tuned as encoders (E5-Mistral, NV-Embed).

Rule of thumb: labels and a fixed label set → encoder. Open-ended output, or no labels → decoder.

# 7. Taking stock

---

# The same estimand, seven times

| era                     | the context map ( $x_{<t} \rightarrow h_t$ ) | what it fixed                                | what stayed broken                |
|-------------------------|----------------------------------------------|----------------------------------------------|-----------------------------------|
| TF-IDF                  | word counts                                  | numbers at all                               | order, meaning, novelty           |
| n-gram                  | last $n-1$ words, a table                    | fluency, a metric                            | window, sparsity, sharing         |
| word vectors            | embeddings + MLP                             | sharing                                      | window, one vector per word       |
| RNN / LSTM              | a running state                              | unbounded window                             | bandwidth, sequential             |
| attention + transformer | every position, retrieved by content         | context-dependent meaning, parallel training | cost $O(T^2)$                     |
| pretraining             | all of the above, trained once on everything | one model, many tasks                        | knowledge cut-off, evidence, cost |

Through all of it: softmax over the vocabulary, cross-entropy, gradient descent. The object never changed:  $p(\text{next word} \mid \text{context})$ .

11:00: what happens when you make the transformer 1,000× bigger, and how a text-completer becomes something you can talk to.

# References

---

- Spärck Jones (1972). A statistical interpretation of term specificity. *J. Documentation*. · Loughran & McDonald (2011). *J. Finance*. · Gentzkow, Kelly & Taddy (2019). Text as data. *JEL*.
- Shannon (1948). A mathematical theory of communication. · Kneser & Ney (1995). Improved backing-off for n-gram LMs. · Chen & Goodman (1999).
- Bengio, Ducharme, Vincent & Jauvin (2003). A neural probabilistic language model. *JMLR*. · Mikolov et al. (2013). *NeurIPS*. · Levy & Goldberg (2014). · Pennington et al. (2014), GloVe.
- Hochreiter & Schmidhuber (1997). LSTM. · Mikolov et al. (2010). RNN LM. · Sutskever, Vinyals & Le (2014). · Cho et al. (2014). · Bahdanau, Cho & Bengio (2015).
- Vaswani et al. (2017). Attention is all you need. · Peters et al. (2018), ELMo. · Radford et al. (2018, 2019), GPT-1/2. · Devlin et al. (2019), BERT. · Reimers & Gurevych (2019), Sentence-BERT. · Huang, Wang & Yang (2023), FinBERT, CAR.
- Hansen, McMahon & Prat (2018). Transparency and deliberation within the FOMC. *QJE*. · Li, Mai, Shen & Yan (2021). Measuring corporate culture using machine learning. *RFS*.

**Textbooks.** Jurafsky & Martin, *Speech and Language Processing*, 3rd ed. ([web.stanford.edu/~jurafsky/slp3](http://web.stanford.edu/~jurafsky/slp3), updated August 2026): ch. 3 (n-grams), 6 (vectors), 8–10 (RNNs, transformers, pretraining). · Raschka, *Build a LLM (From Scratch)*, 2024. · Zhang et al., *Dive into Deep Learning*, d2l.ai, ch. 9–11, 15. · Amidi & Amidi, *Super Study Guide*, 2024, ch. 2–3.